CSCI-570 Homework 2 Solutions

Due Date: Tuesday, September 23rd, 11:59pm

Problem 1.
When discussing the implementation of binary heaps we said that the data is stored in an array. Since we did not impose any limits on the size of the heap, it means that we have to use an unbounded array. However, when discussing the runtime of the operations, we did not consider the cost of re-sizing (copying the entries) the array. Show that the cost of `build` in the online case (i.e. inserting n entries into an empty heap) is $O(n \cdot log\ n)$ even with the cost of copying taken into account. For the sake of simplicity you may assume that we store the root of the heap at index 0, instead of leaving it empty. Provide detailed calculations for the runtime.

[Hint: revisit the cost analysis of unbounded arrays.]

[4 points]

Problem 2.
Suppose you have two binary min-heaps, A and B, with a total of n elements between them. You want to know whether A and B have a key in common. Devise an algorithm that solves this problem in $O(n \cdot log\ n)$ time and besides the two binary heaps it only uses an array of length 2.

[Hint: first come up with a $O(n \cdot log\ n)$ algorithm without the restriction on the size of extra storage.]

[4 points]

Problem 3.
You are given a "k-sorted array": this is an almost-sorted array, where each of the elements are misplaced by less than k positions from their correct location. For example, A = [1,2,3,6,4,5,7] is a 3-sorted array, because the elements 4, 5, and 6 are misplaced by 1, 1, and 2 positions respectively from their correct locations. Design an $O(n \cdot log\ k)$ algorithm to sort the array and analyze the runtime.

[Hint: using a small heap will be helpful.]

[5 points]

Problem 4.
Write a divide and conquer algorithm that finds the largest difference between any two elements of an unsorted array of n numbers.

Example: in case of $A = [3, 7.65, -2, 3, -0.85, 9.15, -1.5]$
the algorithm should return 11.15.

Your algorithm should run in $O(n)$ time. To achieve this, the algorithm needs $T(n) = 2 \cdot T(\frac{n}{2}) + O(1)$ as its runtime recurrence relation. After providing the algorithm, justify that its runtime is indeed $O(n)$ by unrolling its recursion tree.

[5 points]